Joanna Sierra-Mendoza

DSC640: Week 9&10

TSA Complaintsip

Joanna Sierra-Mendoza

DSC640: Week 9&10

TSA Complaints

In [25]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
airport_loc = pd.read_csv("iata-icao.csv")
comp_byairport = pd.read_csv("complaints-by-airport.csv")
comp_bycategory = pd.read_csv("complaints-by-category.csv")
comp_bysubcategory = pd.read_csv("complaints-by-subcategory.csv")

comp_byairport.head()
```

Out[25]:

	pdf_report_date	airport	year_month	count
0	2019-02	ABE	2015-01	0
1	2019-02	ABE	2015-02	0
2	2019-02	ABE	2015-03	0
3	2019-02	ABE	2015-04	0
4	2019-02	ABE	2015-05	2

In [27]:
```
airport_loc.columns
comp_byairport.columns
comp_bycategory.columns
comp_bysubcategory.columns
```

Out[27]:
```
Index(['pdf_report_date', 'airport', 'category', 'subcategory', 'year_month',
       'count', 'clean_cat', 'clean_subcat', 'clean_cat_status',
       'clean_subcat_status', 'is_category_prefix_removed'],
      dtype='object')
```

In [29]:
```
airport_loc.columns
airport_loc.head()
```

Out[29]:

	country_code	region_name	iata	icao	airport	latitude	longitude
0	AE	Abu Zaby	AAN	OMAL	Al Ain International Airport	24.2617	55.6092
1	AE	Abu Zaby	AUH	OMAA	Abu Dhabi International Airport	24.4330	54.6511
2	AE	Abu Zaby	AYM	NaN	Yas Island Seaplane Base	24.4670	54.6103
3	AE	Abu Zaby	AZI	OMAD	Al Bateen Executive Airport	24.4283	54.4581
4	AE	Abu Zaby	DHF	OMAM	Al Dhafra Air Base	24.2482	54.5477

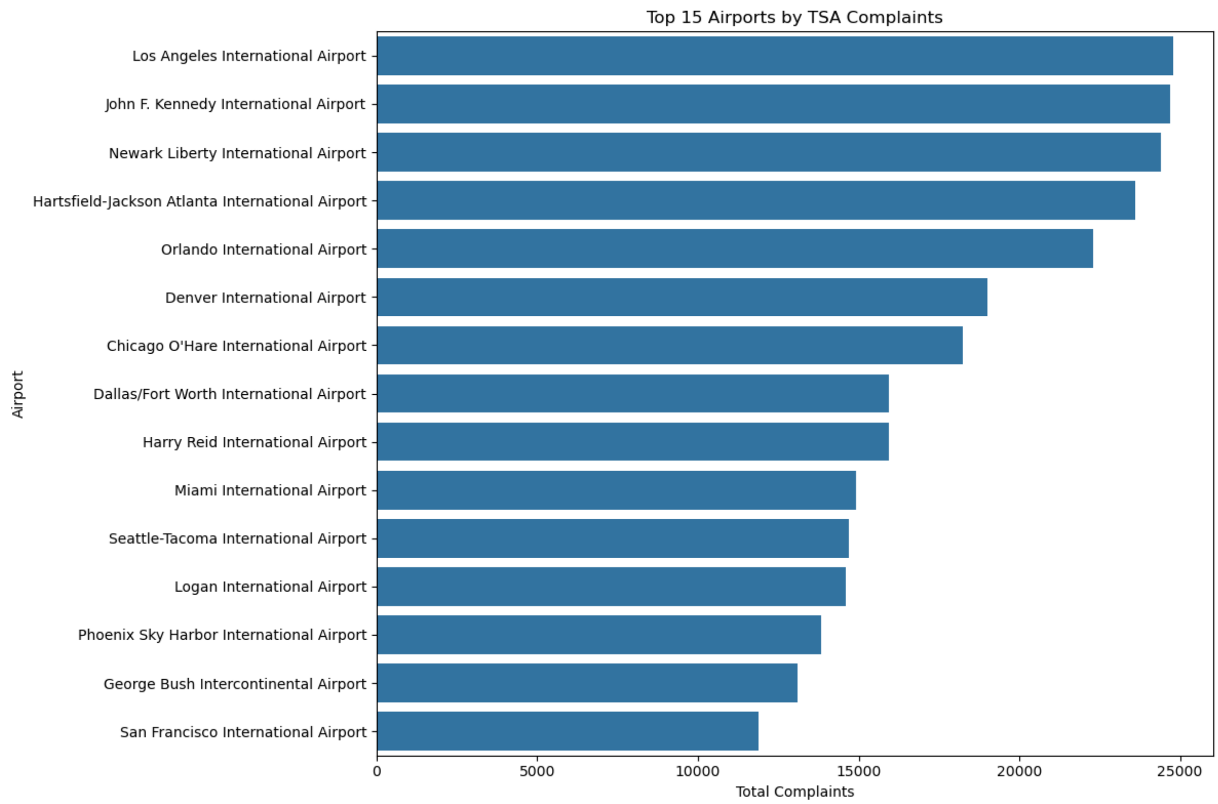
In [31]:
```
#merge airport names and location
airport_loc = airport_loc.rename(columns={
    "iata": "AirportCode",
    "airport": "AirportName"
})

airport_loc = airport_loc[[
    "AirportCode", "AirportName", "country_code", "region_name", "latitude", "longi
]]

comp_byairport = comp_byairport.rename(columns={"airport": "AirportCode"})
comp_bycategory = comp_bycategory.rename(columns={"airport": "AirportCode"})
comp_bysubcategory = comp_bysubcategory.rename(columns={"airport": "AirportCode"})
```

In [33]:
```
comp_byairport_m = comp_byairport.merge(
    airport_loc, on="AirportCode", how="left"
)

comp_bycategory_m = comp_bycategory.merge(
    airport_loc, on="AirportCode", how="left"
)

comp_bysubcategory_m = comp_bysubcategory.merge(
    airport_loc, on="AirportCode", how="left"
)
```

In [41]:
```
#chart1 total complaints over time
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go
comp_byairport_m["year_month"] = pd.to_datetime(comp_byairport_m["year_month"])
ts = (comp_byairport_m
      .groupby("year_month")["count"]
      .sum()
      .reset_index())

plt.figure(figsize=(12, 5))
plt.plot(ts["year_month"], ts["count"], marker="o")
plt.title("Total TSA Complaints Over Time")
```

```
plt.xlabel("Year-Month")
plt.ylabel("Total Complaints")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

In [43]:
```
#total complaints of top airports barchart
top_n = 15
airport_totals = (comp_byairport_m
    .groupby(["AirportCode", "AirportName"], as_index=False)["count"]
    .sum()
)
top_airports = airport_totals.nlargest(top_n, "count")

plt.figure(figsize=(12, 8))
sns.barplot(
    data=top_airports,
    x="count",
    y="AirportName",
    orient="h"
)
plt.title(f"Top {top_n} Airports by TSA Complaints")
plt.xlabel("Total Complaints")
plt.ylabel("Airport")
plt.tight_layout()
plt.show()
```



In [47]:
```
num_categories = comp_bycategory_m["clean_cat"].nunique()
num_categories
comp_bycategory_m["clean_cat"].unique()
```

Out[47]:
```
array(['Hazardous Materials Safety', 'Mishandling of Passenger Property',
       'Civil Rights', 'Customer Service',
       'Identification (ID) Requirements', 'Locks', 'Screening',
       'Patdown', 'Property - Special Handling',
       'Persons w/ Disabilities (PWD)',
       'Expedited Passenger Screening Program', 'Human Capital',
       'Notice of Violation/Civil Penalties',
       'Prohibited & Permitted Items', 'Airline',
       'TSA Contact Information', 'Request for Assistance',
       'Lost and Found', 'No Jurisdiction', 'Claims', 'DHS TRIP',
       'No Subject', 'Duplicate E-mail',
       'Advanced Imaging Technology (AIT)',
       'Additional Information Required/Insufficient Information',
       'Ombudsman', 'Current Topic', 'TWIC',
       'Transfer to Security Specialist', 'Cargo',
       'Intranet/Internet Inquiries',
       'SIDA (Security Identification Display Area)', 'Animals',
       'CSM/Stakeholder Follow-up', 'HAZMAT Endorsement',
       'Freedom of Information Act (FOIA)', 'Flying Armed',
       'Business, Suggestions, and Ideas', 'General Aviation',
       'Federal Flight Deck Officer (FFDO) Program',
       'Transportation Security Operations Center', 'Public Affairs',
       'Call Disconnect/Dropped/Wrong Number',
       'Federal Air Marshall Service (FAMS)', 'System Administrator',
       'SPAM', 'Transcriptions'], dtype=object)
```

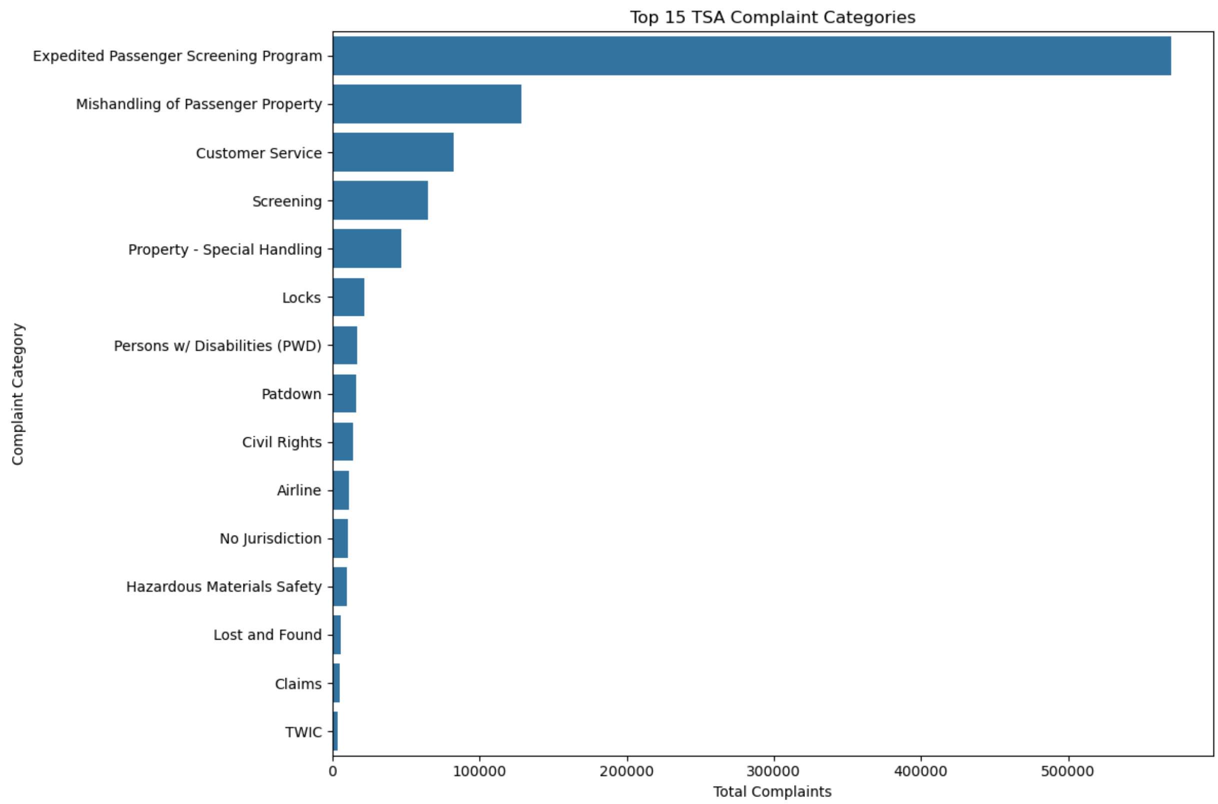
In [49]:
```
num_subcategories = comp_bysubcategory_m["clean_subcat"].nunique()
num_subcategories
comp_bysubcategory_m["clean_subcat"].unique()
```

Out[49]:
```
array(['General', '*Damaged/Missing Items--Checked Baggage',
       'National Origin', 'TSO', 'Domestic', 'Missing or Damaged Lock',
       '*Damaged/Missing Items--Carry-on Luggage',
       'Disarrayed Items--Carry-on or Checked', 'Procedures/Process',
       'Others', '*Damaged/Missing Items', 'Flyer', 'Inconsistency',
       'Racial Profiling or Discrimination', '*Damaged/Missing Items--C',
       '*Damaged Items - Checked', '*Missing Items - Checked',
       'Lengthy Lines', '*Damaged/Missing Items--Unspecified Luggage',
       'Children', 'Medically Necessary Medication and Medical Supplies',
       'Mobility Disability (Not Wheelchair or Scooter)',
       '*Missing Items - Carry-On', '*Damaged Items', '*Missing Item',
       'Security Breach', '*TSA PreCheck',
       'Medical Devices - Attached to the Body', 'Hair Patdowns', 'Race',
       '*', '*TSA Application Program', 'Diabetes',
       'Metal Implants (joint)', 'Wheelchair/ Scooter/ Segway',
       'Flyer as Observer', 'Screening of Religious or Cultural Garments',
       'Prospective Employment with TSA',
       'Diapers/Garments/Dressings/Masks',
       'Prosthesis/Amputees and Casts', '*Damaged Items - Carry-On',
       'Firearm/Ammunition', 'Policy/Regulations/Service',
       'General Complaint and Discrimination', 'Transgender Screening',
       'CSM', 'Autism Spectrum',
       'Cognitive /Developmental /Mental Disability',
       'Crew Member or Airline Employee',
       'Retrieving Abandoned/Confiscated Items',
       'Civil Rights Violation (TSA Employee)',
       'Sex/Gender/Gender Identity- Not Transgender', 'Non-Flyer',
       'Hearing Impairment', 'Service Animals', 'Liquids/Gels/Aerosols',
       'Other', 'Age', 'Sexual Orientation - Not Transgender',
       'Airport/TSA', 'Internal Medical Devices',
       'Touch Aversion (pain/trauma)',
       'Respiratory and Inhaler Equipment', 'Refer to Other',
       'Cancer (general) Not Breast Cancer', 'Sporting Goods',
       'Constitutional Rights', 'TCC', '*Active Duty Military', '*DoD',
       '*TSA/DHS Employees - Opt In', 'Food/Beverages', 'Status',
       'No Referral', 'Ostomy/Ostomates', 'Not Specified',
       '*Known Crew Member', 'Refer to CBP', 'Baggage Requirements',
       'Headwear', '*Damaged/Missing Items Private Screening Contractor',
       'Shoe Screening', 'Perceived Religion', 'Gate Pass',
       'Breast Cancer', 'Lighters/Matches', 'Wounded Warrior (MSIJSOC)',
       'EMAIL ONLY', 'Alcohol and Tobacco',
       'Knives/Tools/Razors/Sharp Objects',
       'ID Matching Ticket and Boarding Pass Issues', 'Current Employee',
       'Beauty & Grooming Items', 'Traveling with Cash',
       'Wounded Warrior', 'Visual Impairment', 'Real ID', 'Lost Baggage',
       'Screening of Religious Cultural and Ceremonial Items',
       'Medication/Prescription/Medically Necessary Items',
       'International Screening', 'Military', 'Assistive Technology',
       '*Over 75', 'International Travel',
       'Breast Milk/Baby Formula/Breast Pump', 'Wheelchair Assistance',
       'Profiling', 'Current Topic', 'Minors Domestic',
       'Refer to Department of State', 'Electronic Devices',
       'Reportable Communication', 'Known Shipper', 'Refer to FAA',
       'Carry-On', 'Classified Materials', 'Request to be Transferred',
       'Chemical/Household Products', 'Remains',
       'Non-Persons w/ Disabilities (PWD)', 'Opposite Gender', 'Checked',
```

```
        '*Children Under 12', 'Rail', 'Marijuana', 'Refer to Airport',
        'Former Employee', 'Ice/Gel Packs',
        'Hostile Work Environment (TSA Employee)',
        'Sexual Harassment (TSA Employee)',
        'Newspaper, Radio, TV Representatives',
        'Indirect Air Carrier (IAC)', 'Cruise Ships',
        'Positive Bag Match Program', 'Knitting/Crochet Hooks/Needles',
        'Oxygen Regulations', 'Alien Flight School Program (AFSP)',
        'Employment Verification', 'TSA Background Check',
        'Transfer to Ombudsman', 'Required Documents',
        'Crewmember Self Defense Training (CMSDT)', 'Claim Form Request',
        'Wage Garnishment', 'Flight Training Security Program (FTSP)'],
       dtype=object)
```

In [55]:

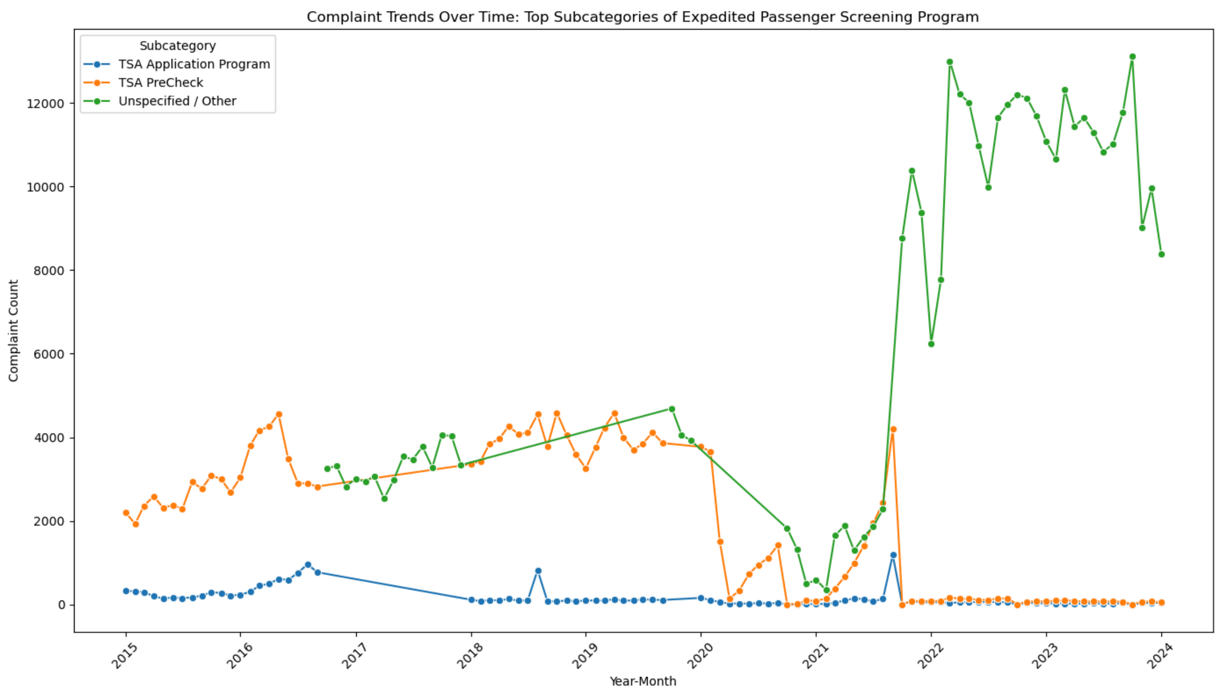
```
#bar chart of complaints by category
cat_totals = (
    comp_bycategory_m
    .groupby("clean_cat", as_index=False)["count"]
    .sum()
    .sort_values("count", ascending=False)
)

top15_cat = cat_totals.head(15)

plt.figure(figsize=(12, 8))
sns.barplot(
    data=top15_cat,
    x="count",
    y="clean_cat",
    orient="h"
)
plt.title("Top 15 TSA Complaint Categories")
plt.xlabel("Total Complaints")
plt.ylabel("Complaint Category")
plt.tight_layout()
plt.show()
```



In [79]:

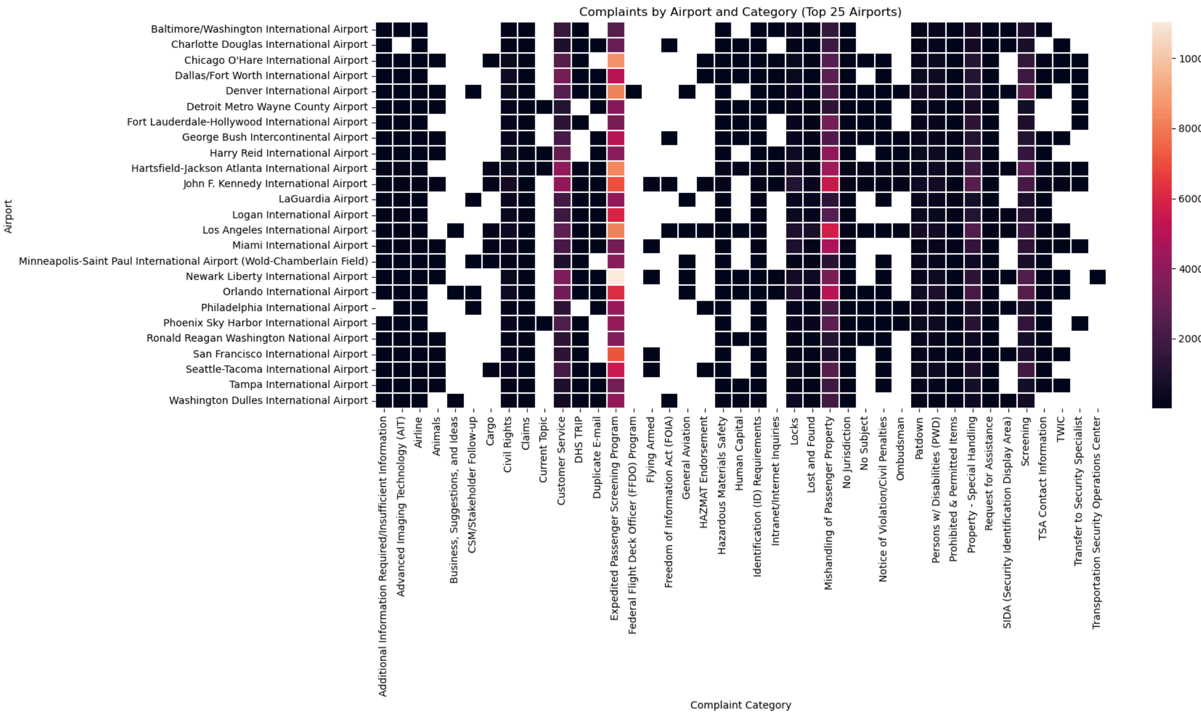
```
#line chart of complaint subcategories for expedited passenger screening program
epsp = comp_bysubcategory_m[
    comp_bysubcategory_m["clean_cat"] == "Expedited Passenger Screening Program"
].copy()

epsp["clean_subcat"] = epsp["clean_subcat"].replace({"*": "Unspecified / Other"})

remove_list = [
    "*Children Under 12",
    "*Over 75",
    "*TSA/DHS Employees - Opt In",
    "*Known Crew Member",
    "*DoD",
    "*Active Duty Military"
]

epsp = epsp[~epsp["clean_subcat"].isin(remove_list)]

epsp["clean_subcat"] = (
    epsp["clean_subcat"]
    .str.replace(r"^\*", "", regex=True)
    .str.strip()
)

epsp["year_month"] = pd.to_datetime(epsp["year_month"])

epsp_time = (
    epsp.groupby(["year_month", "clean_subcat"], as_index=False)["count"]
    .sum()
)
```

```
plt.figure(figsize=(14, 8))
sns.lineplot(
    data=epsp_time,
    x="year_month",
    y="count",
    hue="clean_subcat",
    marker="o"
)
plt.title("Complaint Trends Over Time: Top Subcategories of Expedited Passenger Scr
plt.xlabel("Year-Month")
plt.ylabel("Complaint Count")
plt.legend(title="Subcategory")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



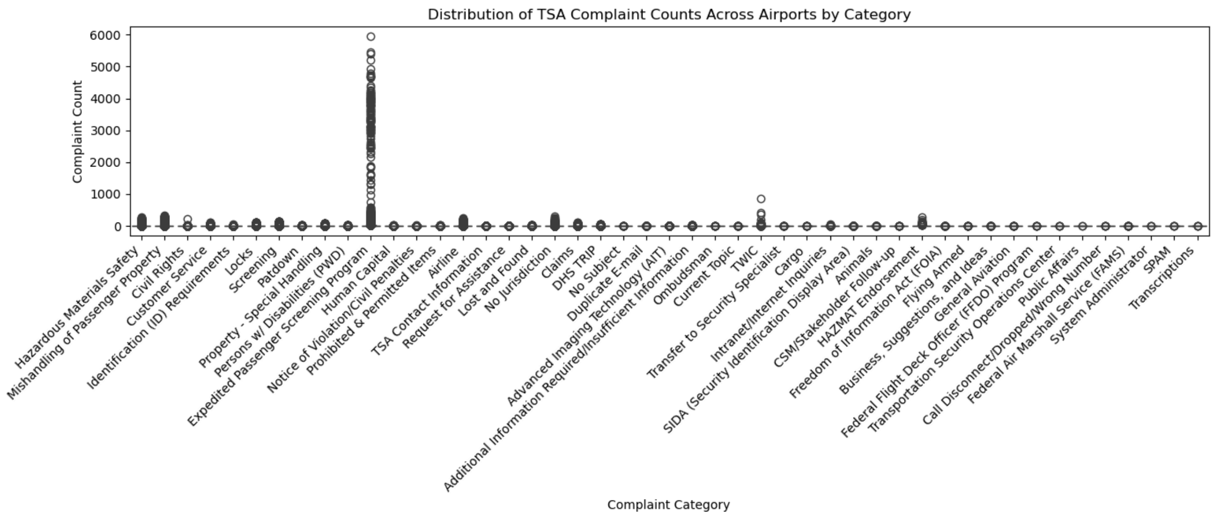
In [91]:
```
#heatmap of top 25 airports and categories
airport_totals = (
    comp_byairport_m.groupby(["AirportCode", "AirportName"], as_index=False)["count
)

top_airports = airport_totals.nlargest(25, "count")["AirportName"]

heat_data = comp_bycategory_m[comp_bycategory_m["AirportName"].isin(top_airports)]

heat_pivot = heat_data.pivot_table(
    index="AirportName",
    columns="clean_cat",
    values="count",
    aggfunc="sum"
)

plt.figure(figsize=(18, 10))
sns.heatmap(heat_pivot, linewidths=0.3)
plt.title("Complaints by Airport and Category (Top 25 Airports)")
```

```
plt.xlabel("Complaint Category")
plt.ylabel("Airport")
plt.tight_layout()
plt.show()
```



In [93]:

```
#boxplot of tsa complaints
plt.figure(figsize=(14, 6))

sns.boxplot(
    data=comp_bycategory_m,
    x="clean_cat",
    y="count",
)

plt.title("Distribution of TSA Complaint Counts Across Airports by Category")
plt.xlabel("Complaint Category")
plt.ylabel("Complaint Count")

plt.xticks(rotation=45, ha="right")

plt.tight_layout()
plt.show()
```



In [97]:

```
#geo dataset
epsp_airports = comp_bycategory_m[
    comp_bycategory_m["clean_cat"] == "Expedited Passenger Screening Program"
].copy()

epsp_geo = (
    epsp_airports.groupby(
        ["AirportCode", "AirportName", "latitude", "longitude"], as_index=False
    )["count"].sum()
)
```

In [107…

```
top_epsp = epsp_geo.nlargest(30, "count")
fig = px.scatter_geo(
    top_epsp,
    lat="latitude",
    lon="longitude",
    size="count",
    color="count",
    hover_name="AirportName",
    scope="north america",
    title="ESPS Complaint Map"
)

fig.show()
```

In [ ]:

```

```